Approval-Based Elections in the 1D Euclidean Domain

Jan Sznajd

Supervisor: dr hab. inż. Piotr Faliszewski

Department of Computer Science
AGH University of Science and Technology

This dissertation is submitted for the degree of
Master of Science

February 2021

TODO

0.1 My Questions

https://github.com/jasieksz/master-thesis-code/blob/master/results/profiles.md

1. ====
2. Big M parameter in the detection ILP,
3. VR/CR MAV Liu and Guo (2016) - upper bound distance d
4. $hammingDistance(vectorK, A[voterIndex]) + k - len(K) > d$

0.1.1 VR/CR MAV Liu and Guo (2016) - upper bound distance d

Profile

	A	B	C	D
0	0	0	1	0
1	1	0	1	0
2	1	1	0	1
3	1	0	0	1

Brute Force Results

```
      MAV, Committee
k=1     2, [A]
        3, [B], [D]
        4, [C]

k=2     3, [A, B], [A, C], [A, D], [C, D]
        4, [B, C], [B, D]

k=3     2, [A, C, D]
        3, [A, B, C], [B, C, D]
        4, [A, B, D]
```

Greedy MAV (Liu and Guo (2016)) Results

candidates after || are the ones selected arbitrarily if K is too small after algo. finished.

```
k=1
    d=3 [D]
    d=4 [ || A]

k=2
    d=3 [C, D]
    d=4 [D || A]
    d=5 [ || A, B]

k=3
```

```
d=2 [C, D || A]
d=3 [C, D || A]
d=4 [C, D || A]
d=5 [D || A, B]
d=6 [ || A, B, C]
```

Brute Force

```
def mavScore(A: np.ndarray, committee: np.array) -> int:
    partialHD = lambda preference: hammingDistance(committee, preference)
    return max([partialHD(pref) for pref in A])

def minimax(A: np.ndarray, k: int) -> np.array:
    candidateCount = A.shape[1]
    scores = [(mavScore(A, committeeTupleToVector(K, candidateCount)), K)
                for K in combinations(range(candidateCount), k)]
    scores.sort(key=lambda mavScore_committee: mavScore_committee[0])
    return scores
```

Greedy MAV (Liu and Guo (2016))

```
def minimaxCR(A, k, d):
    n,m = A.shape # Vn Cm
    C = [i for i in range(m)]
    K = []
    for voterIndex in range(n):
        X = [candidate for candidate in C if candidate not in K]
        vectorK = committeeTupleToVector(committee=K, C=m)
        s = hammingDistance2(vectorK, A[voterIndex]) + k - len(K)
        if s > d:
            r = math.ceil((s - d) / 2)
            if r > k - len(K) or r > len(X):
                print("NOT FOUND")
                return []

            X_sorted = getCandsSortedByVotes(A=A, X=X)
```

```
            X_after = getCandsAfter(C=X_sorted, after=voterIndex)
            for c in X_after[:r]:
                K.append(c)
    if k > len(K):
        print("ARBITRARY SELECTION - Current K = {}".format(K))
        for c in [c for c in C if c not in K][:(k-len(K))]:
            K.append(c)
    return K
```

0.2 10/12/2020

1. Refactor code - vcr ilp detection, profile generation
2. Generate and analyze profiles: 5x5, 4x6, 6x4, random NxM

0.3 17/12/2020

1. Implement ilp MAV
2. Implement VR/CR MAV Liu and Guo (2016)
3. Compare ilp and poly MAV

0.4 07/01/2021

1. VR/CR MAV Liu and Guo (2016)

0.5 14/01/2021

1. VR/CR MAV Liu and Guo (2016)

0.6 11/02/2021

1. write thesis (+- 3 pages)
2. email about CR MAV to Liu/Guo

3. fix CR MAV algo
4. try to proof CR MAV
5. single thread experiments

Abstract

(WIP)

I study approval-based elections, where preferences are in the 1D-Euclidean domain. I explore multiple areas, notably:

1. I create domain detection algorithms and evaluate the results.
2. I look for efficient method of computing election results.

For the first issue I find statistical relation with other well known domains. For the second one I create polynomial complexity algorithm and compare it with LP solutions. (WIP)

Table of contents

TODO **ii**
0.1 My Questions ii
0.1.1 VR/CR MAV Liu and Guo (2016) - upper bound distance d iii
0.2 10/12/2020 v
0.3 17/12/2020 v
0.4 07/01/2021 v
0.5 14/01/2021 v
0.6 11/02/2021 v

1 Introduction **1**

2 Preliminaries **2**

3 Approval-Based Elections in the *1D* Euclidean Domain **4**
3.1 Euclidean Preference Restrictions 4
3.1.1 VR and CR Detection Algorithms 5
3.1.2 VCR Detection Algorithms 5
3.2 (WIP) Committee Scoring Rules - Motivation 6

4 Solution **7**
4.1 Voter Candidate Range Profile Detection 7
4.1.1 Integer Linear Programming 7
4.1.2 Polynomial Complexity VCR detection algorithm 9
4.1.3 Performance Comparison 9
4.2 Voter Candidate Range Profiles 10
4.2.1 (WIP) Experiment description - search of domain 10
4.2.2 How many VCR profiles exists that are not VR nor CR? 10
4.2.3 (WIP) Experiment conclusions 11
4.3 Minimax Scoring Rule Algorithms 12

4.3.1 Overview 12
4.3.2 Polynomial algorithm for VCR profiles 12
4.3.3 Comparison 13
4.4 WIP: Approval deletion for VCR profile to become VR or CR 14

5 Evaluation **15**
5.1 Conclusions 15
5.2 Future work 15

References **16**

Chapter 1

Introduction

Elections are ubiquitous element of our society in one form or another. For example choosing members of parliament or a winner of a competition. In a day-to-day activities we need to decide on a meeting time or select recommended movies in a streaming service. Each of these choices may require different type of election and scoring rule, based on the underlying properties of the choice (i.e. voters should be represented proportionally by members of a government)

Reseach objectives

In this work I am going to focus on the *Approval Elections*, that is where each voter provides a subset of candidates that he approves of and disapproves the rest. In general many social choice problems are known to be computationally hard. However they become tractable under some restrictions on the structure of the preferences. Elkind and Lackner (2015) explored well known restrictions in the setting of approval preferences, introduced several new one-dimensional domain restrictions such as Dichotomous Euclidean and derivatives (czy to dobre okreslenie na VR CR?). Faliszewski (2020) generalized this model for the N-dimensional euclidean space calling it Voter Candidate Range and explored two-dimensional case.

The goal of this work is to explore properties of the one-dimensional model. Search for the profile detection polynomial complexity algorithm, however Peters (2017) proved this to be NP-hard for the ordinal elections. I will try as well to extend solution of Liu and Guo (2016) for computing MAV score with polynomial complexity to the VCR model.

Contributions

I provide fast linear programming preference detection algorithm and ...

Chapter 2

Preliminaries

In this chapter, I introduce the definitions and notations used in this work.

Approval Elections

Election is a pair $E = (C, V)$ where $C = \{c_1, ..., c_m\}$ is a set of candidates and $V = \{v_1, ..., v_n\}$ is a list of voters. Each voter $v \in V$ approves a subset of the candidates and disapproves the remaining candidates, I will be referring to it as a *approval set* and for each voter $v \in V$ denote it as $A(v)$, when using A just by itself I refer to all *approval set* in the given election.

Committee Selection Rules

Voting rules are algorithms used to select winners (committee) of a given election. Each rule has different properties and must be selected accordingly to the goal of the election. For the detailed overview of multiwinner voting rules I point to Faliszewski et al. (2017).

Hamming Distance Minimax Approval Voting (MAV)

Selects a committee W that minimizes the maximum Hamming distance between W and voters' approval ballots. For an election E and a resulting committee of size k I denote winning committee as $W = MAV(E, k)$ Brams et al. (2007)

Domain restrictions

Dichotmous Euclidean, Voter Interval, Candidate Interval

Elkind and Lackner (2015)

Single Peaked

Single Peaked On a Circle - Peters and Lackner (2020)

Single Crossing

Chapter 3

Approval-Based Elections in the *1D* Euclidean Domain

3.1 Euclidean Preference Restrictions

Voter Candidate Range

Given an election $E = (C, V)$, we say that the voters have *1D-Euclidean* preferences For each agent there exists a point x_a and nonnegative real value r_a such that: $c \in A(v) \Leftrightarrow \sqrt{(x_c - x_v)^2} \leq r_c + r_v$

Intuitively, for $a \in C \cup V$ the point x_a describes a's ideal position in 1-dimensional space of opinions. For candidate $c \in C, r_c$ can be seen as c's charisma. For candidate $v \in V, r_v$ can be seen as v's willingness to compromise. Faliszewski (2020)

Candidate Range

We say that P satisfies CR if candidates in C can be ordered so that each of the sets vi forms an interval of that ordering. CR is special case of VCR, where we require all the voters to have radii equal to zero

Voter Range

We say that P satisfies VR if the voters in P can be reordered so that for every candidate c the voters that approve c form an interval of that ordering. VR is a special case of VCR, where we require all the candidates to have radii equal to zero

Faliszewski (2020) showed that **CR** and **VR** are equivalent to respectively **VI** and **CI**, introduced by Elkind and Lackner (2015).

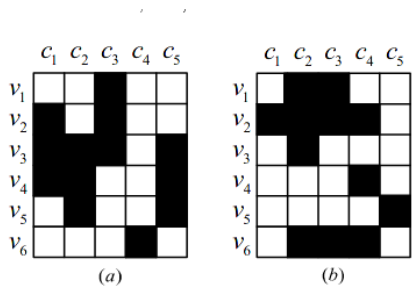

Fig. 3.1 VR and CR Temporary image

Larger Domain

The VCR model is strictly more powerful than the VR and CR ones. For example, election with candidate set $C = \{a,b,c,d\}$ and voters with approval sets $\{a,b\}$, $\{b,c\}$, $\{b,d\}$, and $\{a,b,c,d\}$ is VCR, but neither VR nor CR.

3.1.1 VR and CR Detection Algorithms

Profile is VR or CR if the matrix or any permutation has consecutive ones property, that is for each row (VR) or column (CR), entries with 1 are in the uninterrupted sequence.

The fastest known algorithm for detection of C1P is based on the *PQ-tree* algorithms by Booth and Lueker (1976). However in this work I'm using still relatively fast and much easier to implement approach based on the *Lex-BFS* algorithm by Habib et al. (2000)

(WIP) Change of the solution, using ILP instead - described in the ch.4

3.1.2 VCR Detection Algorithms

(WIP) change titles of subsections and merge them together?

Voter Candidate Range detection algorithms are described in detail in the subsequent chapters.

3.2 (WIP) Committee Scoring Rules - Motivation

Explain how VR/CR/VCR can change complexity of computing rules, detection, etc.

Section 4.1 in Faliszewski (2020)

"Elkind and Lackner (2015) have shown polynomial-time algorithms for computing CC winning committees in 1D-VR and 1D-CR elections. We unify this result into a single algorithm for 1D-VCR elections (the main idea is remarkably close to that for the 1D-VR case). Proposition 2. There is an algorithm that given a 1D-VCR election and committee size, computes some winning C committee in polynomial time. A natural question is whether it is possible to extend the above result to PAV and other Thiele methods. While we leave this question open, in Appendix B we explain that the existing technique that allowed to construct a polynomial-time algorithm for a large class of Thiele rules under the 1D-VR model (Peters and Lackner 2020) cannot be easily applied to the 1D-VCR setting. On the other hand, it may be possible to obtain a 1D-VCR algorithm for MAV as 1D-VR and 1D-CR algorithms do exist (Liu and Guo 2016)."

Chapter 4

Solution

4.1 Voter Candidate Range Profile Detection

This section describes in detail and compares created algorithms for the VCR property detection.

4.1.1 Integer Linear Programming

The only input is *approval set* of each voter. Represented by $2D$ matrix A_{vc}

Variables

x_c - candidate c position on x axis
x_v - voter v position on x axis
r_c - candidate c radius
r_v - voter v radius

Transform VCR condition to linear constraints

VCR definition formula contains square root and power elements (WIP: Inne słowo). Hence it's not directly suitable for ILP formulation. Let's transform it to linear constraint.

$$
\begin{aligned}
VCR = & \\
&= \sqrt{(x_c - x_v)^2} \leq r_c + r_v \\
&= \|x_c - x_v\| \leq r_c + r_v \\
&= -(r_c + r_v) \leq x_c - x_v \leq r_c + r_v \\
&\iff x_c - x_v \leq r_c + r_v \;\; \vee \;\; -(x_c - x_v) \leq r_c + r_v
\end{aligned}
\tag{4.1}
$$

Big M method

Above constraint contains $\vee$ condition which cannot be represented directly in the linear programming. I used the *BigM* method to solve this issue

(WIP) Fix explanation, based on :

When used in the constraints themselves, one of the many uses of Big M, for example, refers to ensuring equality of variables only when a certain binary variable takes on one value, but to leave the variables "open" if the binary variable takes on its opposite value. One instance of this is as follows: for a sufficiently large M and z binary variable (0 or 1), the constraints

$$x - y \leq Mz \tag{4.2}$$

$$x - y \geq -Mz \tag{4.3}$$

ensure that $z = 0 \rightarrow x = y$, otherwise $z = 1 \rightarrow -M \leq x - y \leq M$ indicating that the variables x and y can have any values so long as the absolute value of their difference is bounded by M (hence the need for M to be "large enough.")

ILP detection algorithm

Introduce large constant positive parameter M
Introduce new binary variables for each pair of candidate and voter:
$z1_{cv}$
$z2_{cv}$

$$
\begin{aligned}
&\textit{Minimize 0}\\
&\textit{foreach (c,v) in } C \times V :\\
&\textit{if } c \in ApprovalSet(v) \textit{ add constraints}\\
&x_c - x_v \leq r_c + r_v\\
&-(x_c - x_v) \leq r_c + r_v\\
&\textit{if } c \notin ApprovalSet(v) \textit{ add constraints}\\
&x_c - x_v + M(1 - z1_{cv}) \geq r_c + r_v + 1\\
&-(x_c - x_v) + M(1 - z2_{cv}) \geq r_c + r_v + 1\\
&z1_{cv} + z2_{cv} >= 1
\end{aligned}
$$

(WIP) : Rewrite solution using just one binary var

4.1.2 Polynomial Complexity VCR detection algorithm

4.1.3 Performance Comparison

4.2 Voter Candidate Range Profiles

In this section I describe exploration of VCR domain and found properties.

4.2.1 (WIP) Experiment description - search of domain

Experiments can be grouped into 2 categories. First search of small profiles, where I check every combination, and second one for larger profiles where I generate random preferences.

Algorithm steps:

1. Generate approval sets of the profiles for given C and V elections.
2. Filter out profiles not fulfilling *VCR* property. Described in the section 4.1
3. Group profiles by *VR, CR* and *VCR not VR nor CR* properties (*VCR !COP*)
4. Aggregate statistics

To achieve high performance I used *MapReduce* paradigm with *Spark*, as the tasks are independent and easy to parallelize.

4.2.2 How many VCR profiles exists that are not VR nor CR?

Search of all the small profiles

For the smaller size of profiles I performed search of all the profile combinations.

	3C 3V	4C 4V	5C 5V	4C 6V	6C 4V
Count	2^9	2^{16}	2^{25}	2^{24}	2^{24}
VCR	506	57832	19846082	10893736	10893736
VR	-	-	-	-	-
CR	-	-	-	-	-
VCR !COP	0	96	501000	144480	144480

WIP : Prettify the table
WIP : Make some charts, write statistics, show examples

Profile examples

Example of 5C 5V elections that are VCR but not VR nor CR

```
A=[
    [1., 1., 0., 0., 1.],
    [1., 1., 1., 1., 1.],
    [1., 0., 0., 0., 0.],
    [0., 0., 0., 1., 1.],
    [0., 0., 1., 0., 1.]],
C=[
    Candidate(id='A', x=3.0, r=1.0),
    Candidate(id='B', x=2.0, r=0.0),
    Candidate(id='C', x=0.0, r=0.0),
    Candidate(id='D', x=1.0, r=0.0),
    Candidate(id='E', x=1.0, r=1.0)],
V=[
    Voter(id='V0', x=2.0, r=0.0),
    Voter(id='V1', x=0.0, r=2000000000.0),
    Voter(id='V2', x=3.0, r=0.0),
    Voter(id='V3', x=1.0, r=0.0),
    Voter(id='V4', x=0.0, r=0.0)])]
```

Search of the random large profiles

For the larger size of profiles I generated a suitable number of random profiles.

	20C 20V	60C 60V	100C 100V
Count	400000	-	-
VCR	-	-	-
VR	-	-	-
CR	-	-	-
VCR !COP	-	-	-

4.2.3 (WIP) Experiment conclusions

Maybe do some nice graphs here and explore properties

4.3 Minimax Scoring Rule Algorithms

4.3.1 Overview

Minimax Approval Voting (MAV) As close as possible to the preferences of all parties. By "close" we mean that the maximum distance of the compromise from the position of any state is minimal, which we call a minimax outcome. Brams et al. (2007) Selects a committee W that minimizes the maximum Hamming distance between W and voters' approval ballots. We define MAV-score of a committee W as $Max(f(W,v1),...,f(W,vn))$ where $f(W,vi) =$.

(WIP) Rewrite MAV definition, use easier equation formulation

MAV returns a committee W with the lowest MAV-score.

Brute force algorithm

```
def hammingDistance(a: np.array, b: np.array) -> int:
return (a != b).nonzero()[0].shape[0]

def mavScore(profile: Profile, committee:np.array) -> int:
    partialHD = lambda preference: hammingDistance(committee, preference)
    return max([partialHD(pref) for pref in profile.A])

def minimax(profile: Profile, k: int) -> np.array:
    candidateCount = len(profile.C)
    scores = [(mavScore(profile, np.array(committee)), profile)
                for committee in combinations(range(candidateCount), k)]
    scores.sort(key=lambda mav_p: mav_p[0])
    return scores
```

Implementation of ILP from Lackner and Skowron (2020)

Implementation of Poly for VR and CR from Liu and Guo (2016)

4.3.2 Polynomial algorithm for VCR profiles

(WIP) Current approches

Greedy

1. Sort by ?

2. ?

Dynamic

4.3.3 Comparison

4.4 WIP: Approval deletion for VCR profile to become VR or CR

Chapter 5

Evaluation

5.1 Conclusions

5.2 Future work

References

Booth, K. S. and Lueker, G. S. (1976). Testing for the consecutive ones property, interval graphs, and graph planarity using pq-tree algorithms. *Journal of Computer and System Sciences*, 13(3):335 – 379.

Brams, S., Kilgour, D., and Sanver, M. R. (2007). *A Minimax Procedure for Negotiating Multilateral Treaties*, pages 265–282.

Elkind, E. and Lackner, M. (2015). Structure in dichotomous preferences. *CoRR*, abs/1505.00341.

Faliszewski, P. (2020). An analysis of approval-based committee rules for 2d-euclidean elections.

Faliszewski, P., Skowron, P., Slink, A., and Talmon, N. (2017). *Multiwinner Voting: A New Challenge for Social Choice Theory*. AI Access.

Habib, M., McConnell, R., Paul, C., and Viennot, L. (2000). Lex-bfs and partition refinement, with applications to transitive orientation, interval graph recognition and consecutive ones testing. *Theoretical Computer Science*, 234(1):59 – 84.

Lackner, M. and Skowron, P. (2020). Approval-based committee voting: Axioms, algorithms, and applications.

Liu, H. and Guo, J. (2016). *Parameterized Complexity of Winner Determination in Minimax Committee Elections*, page 341–349. AAMAS '16. International Foundation for Autonomous Agents and Multiagent Systems.

Peters, D. (2017). Recognising multidimensional euclidean preferences. In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, AAAI'17, page 642–648. AAAI Press.

Peters, D. and Lackner, M. (2020). Preferences single-peaked on a circle. *J. Artif. Intell. Res.*, 68:463–502.